CORS + RACK-CORS MIT RAILS, GRAPHQL & VITE

Deutsche Fassung - Inhalt 1:1, mit Inhaltsverzeichnis

INHALTSVERZEICHNIS

1. Was ist CORS?
2. bundle add rack-cors
3. Request Flow
4. Rails Development Config
5. OPTIONS / Preflight
6. CORS vs Authentication
7. Devise Session Cookie
8. Credentials
9. Produktion
10. Vite und rack-cors
11. Vite Proxy
12. Gesamtarchitektur
13. Zusammenfassung - Code

1. Was ist CORS?

CORS (Cross-Origin Resource Sharing) ist ein Browser-Mechanismus, der entscheidet, ob ein Frontend die Response eines Backends mit anderer Origin lesen darf. In Development läuft Vite oft auf http://localhost:5173 und Rails auf http://localhost:3000; unterschiedliche Ports bedeuten unterschiedliche Origins.

2. Wofür ist `bundle add rack-cors`?

Dieser Befehl wird im Rails-Projekt ausgeführt. Er fügt das Gem rack-cors zur Gemfile hinzu und installiert eine Rack-Middleware für CORS. `rack-cors` ist kein Vite-Paket und wird nicht mit npm installiert.

```
bundle add rack-cors

# Gemfile
gem "rack-cors"
```

3. Request Flow

rack-cors sitzt vor dem Rails Controller. Die Middleware prüft die Origin und ergänzt die nötigen CORS Response Headers, bevor der Browser dem Frontend erlaubt, die Response zu lesen.

```
Vite :5173
    |
    | POST /graphql
    v
Rack::Cors
    |
    v
Rails
    |
    v
GraphqlController
    |
    v
GraphQL Schema
```

4. Rails-Konfiguration in Development

config/initializers/cors.rb anlegen und nur der Vite-Origin Zugriff auf /graphql erlauben.

```
Rails.application.config.middleware.insert_before 0, Rack::Cors do
  allow do
    origins "http://localhost:5173"

    resource "/graphql",
      headers: :any,
      methods: [:post, :options]
  end
end
```

5. Warum OPTIONS?

Bei bestimmten Cross-Origin Requests sendet der Browser zuerst einen Preflight-Request mit OPTIONS. Damit fragt er, ob Origin, HTTP-Methode und Headers erlaubt sind. rack-cors verarbeitet diesen Preflight.

```
OPTIONS /graphql
Origin: http://localhost:5173

        |
        v
     CORS OK
        |
        v
POST /graphql
```

6. CORS ist keine Authentication

CORS entscheidet nur, ob der Browser Cross-Origin-Kommunikation erlaubt. Devise bestimmt, wer der Benutzer ist; eine Policy bestimmt, was der Benutzer tun darf. Erfolgreiches CORS bedeutet nicht, dass ein Benutzer ein Project lesen oder einen Task ändern darf.

```
CORS
-> Darf der Browser das Backend aufrufen?

Devise
-> Wer ist der Benutzer?

Policy
-> Was darf der Benutzer tun?
```

7. Devise Session Cookie

Wenn Vite und Rails Devise Session Cookies verwenden, braucht das Frontend typischerweise credentials: "include" und CORS muss Credentials erlauben. Bei Credentials muss eine konkrete Origin konfiguriert werden.

```
fetch("http://localhost:3000/graphql", {
  method: "POST",
  credentials: "include",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ query, variables })
})
```

8. Rails CORS mit Credentials

Beispielkonfiguration für Devise Session Cookies:

```
Rails.application.config.middleware.insert_before 0, Rack::Cors do
  allow do
    origins "http://localhost:5173"

    resource "/graphql",
      headers: :any,
      methods: [:post, :options],
      credentials: true
  end
end
```

9. Produktion

Nicht standardmäßig origins "*" verwenden, besonders nicht mit Credentials/Cookies. FRONTEND_URL aus der Umgebung verwenden, damit Development und Produktion getrennte Origins haben.

```
origins ENV.fetch(
  "FRONTEND_URL",
  "http://localhost:5173"
)

# development
FRONTEND_URL=http://localhost:5173

# production example
FRONTEND_URL=https://app.example.com
```

10. Braucht Vite `rack-cors`?

Nein. rack-cors gehört zu Rails/Rack. Vite benötigt kein npm install rack-cors. bundle add rack-cors wird im Rails-Projektverzeichnis ausgeführt.

```
Rails
Gemfile
└── rack-cors

Vite
package.json
└── kein rack-cors
```

11. Vite Proxy in Development

Alternativ kann Vite /graphql zu Rails proxyen. Der Browser ruft /graphql auf der Vite-Origin auf und Vite leitet den Request an localhost:3000 weiter. Das vereinfacht Development, ersetzt aber kein korrektes Production-CORS, wenn Frontend und API in Produktion tatsächlich unterschiedliche Origins haben.

```
// vite.config.js
export default {
  server: {
    proxy: {
      "/graphql": {
        target: "http://localhost:3000",
        changeOrigin: true
      }
    }
  }
}

// frontend
fetch("/graphql", ...)
```

12. Gesamtarchitektur

Im GraphQL-Rails-Vite-Devise-Projekt ist CORS nur die Kommunikationsschicht zwischen Browser und Backend; Authentication und Authorization liegen weiterhin dahinter.

```
Vite :5173
    |
    v
CORS / rack-cors
    |
    v
Rails :3000
    |
    v
Devise / current_user
    |
    v
GraphQL
    |
    v
Policy
    |
    v
Project / Task
```

Zusammenfassung - Code

Kurze Konfiguration zum Merken:

```
# Rails project
bundle add rack-cors

# config/initializers/cors.rb
Rails.application.config.middleware.insert_before 0, Rack::Cors do
  allow do
    origins ENV.fetch(
      "FRONTEND_URL",
      "http://localhost:5173"
    )

    resource "/graphql",
      headers: :any,
      methods: [:post, :options],
      credentials: true
  end
end

# Vite / Browser
fetch("http://localhost:3000/graphql", {
  method: "POST",
  credentials: "include",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ query, variables })
})

CORS != Authentication
CORS != Authorization
rack-cors gehört zu Rails, nicht zu Vite.
```